
Documentation technique

Flixy Connect

ROCHER Maxime

BTS SIO

Option SLAM

2025-2026

Table des matières

1 Introduction.....	4
1.1 Objectif du document.....	4
1.2 Contexte du projet.....	4
2 Présentation projet.....	5
2.1 Présentation générale.....	5
2.2 Objectifs du projet.....	5
3 Environnement développement.....	6
3.1 Environnement logiciel.....	6
3.2 Technologies utilisées.....	6
3.3 Gestion de version.....	6
4 Architecture projet.....	7
4.1 Architecture Générale.....	7
4.2 Structure du Projet.....	7
4.3 Couche UI.....	8
4.4 Couche ViewModel.....	8
4.5 Couche Repository.....	9
4.6 Couche Data.....	9
4.7 store.....	10
4.8 Couche Network.....	10
4.9 Couche Utils.....	10
4.10 Worker.....	11
4.11 Activité principale.....	11
5 Base de données.....	12
5.1 Relations.....	13
6 API utilisées.....	14
7 Fonctionnalités techniques.....	15
7.1 Authentification Utilisateur.....	15
7.1.1 Présentation.....	15
7.1.2 Structure de la fonctionnalité.....	16
7.1.3 Modèle utilisateur (UserEntity).....	16
7.1.4 Inscription utilisateur.....	17
7.1.5 Connexion utilisateur.....	19
7.1.6 UserViewModel.....	20
7.1.7 Gestion session utilisateur.....	20
7.1.8 Sécurité.....	21
7.2 Profil Utilisateur.....	22
7.2.1 Présentation.....	22
7.2.2 Chargement des données.....	23
7.2.3 Interface utilisateur.....	23
7.2.4 Gestion des commentaires.....	24
7.2.5 Notifications.....	25

7.2.6 Déconnexion.....	25
7.2.7 Observations techniques.....	25
7.3 Gestion des films et API TMDb.....	26
7.3.1 Présentation.....	26
7.3.2 Architecture de la fonctionnalité.....	26
7.3.3 Modèle Film (MovieEntity).....	27
7.3.4 Récupération des films via l'API.....	27
7.3.5 Affichage des films.....	28
7.4 Notifications planifiées (WorkManager).....	28
7.4.1 Présentation.....	28
7.4.2 Architecture.....	28
7.4.3 Worker et Notification.....	29
7.4.4 Scheduler (NotificationScheduler).....	29
7.4.5 Gestion des préférences.....	29
7.4.6 Ajout d'un nouveau type de notification.....	30
8 Schémas techniques.....	31
8.1 Diagramme UML.....	31
8.2 Diagramme base de données.....	33
9 Gestion erreurs.....	35
9.1 Gestion des erreurs API.....	35
9.2 Gestion des exceptions.....	35
9.3 Gestion des états de chargement.....	36
9.4 Gestion des erreurs d'authentification.....	36
9.5 Gestion globale des erreurs.....	36
10 Sécurité.....	37
10.1 Sécurisation des données utilisateur.....	37
10.2 Vérification sécurisée lors de la connexion.....	37
10.3 Stockage local sécurisé.....	38
10.4 Architecture MVVM et Sécurité.....	38
10.5 Gestion de session utilisateur.....	39
10.6 Sécurité Base de Données Locale.....	39
10.7 Sécurité globale mise en place.....	39
11 Limites.....	40
12 Évolutions.....	41
12.1 Système d'amitiés entre utilisateurs.....	41
12.2 Partage de contenus entre amis.....	41
12.3 Notifications.....	42
12.4 Synchronisation des données.....	42
13 Conclusion.....	43

1 Introduction

1.1 Objectif du document

Cette documentation technique a pour objectif de présenter l'ensemble des éléments techniques liés au développement de l'application mobile Flixy Connect, réalisée pour l'entreprise Flixy Studios.

Ce document décrit l'architecture générale du projet, les technologies utilisées, la gestion des données ainsi que les différentes fonctionnalités techniques mises en place dans l'application.

Cette documentation est destinée :

- Aux développeurs souhaitant maintenir ou améliorer l'application
- Aux équipes techniques de l'entreprise Flixy Studios
- Aux évaluateurs dans le cadre du BTS SIO

L'objectif est de fournir une vision claire de l'organisation technique du projet afin de faciliter sa compréhension, sa maintenance et son évolution.

1.2 Contexte du projet

Dans un contexte marqué par l'évolution des usages numériques dans le domaine du divertissement, l'entreprise Flixy Studios souhaite proposer une application mobile permettant aux utilisateurs de consulter facilement des contenus cinématographiques.

Afin de répondre à ce besoin, l'application Flixy Connect a été développée. Cette application permet aux utilisateurs de consulter un catalogue de films, d'accéder à des informations détaillées et d'interagir avec les contenus disponibles.

L'application repose sur l'utilisation d'une API externe permettant de récupérer les données relatives aux films ainsi que sur une base de données locale permettant de stocker les données utilisateur.

2 Présentation projet

2.1 Présentation générale

Le projet Flixty Connect est une application mobile Android développée pour l'entreprise Flixty Studios, spécialisée dans l'exploitation cinématographique.

L'application permet aux utilisateurs de consulter un catalogue de films à partir d'une API externe et d'interagir avec ces contenus grâce à différentes fonctionnalités.

Les principales fonctionnalités de l'application sont :

- Consultation du catalogue de films
- Consultation des détails d'un film
- Ajout d'un film aux favoris
- Notation des films
- Publication de commentaires

Les données relatives aux films sont récupérées via l'API TMDb (The Movie Database).

2.2 Objectifs du projet

L'objectif principal de l'application est de proposer une solution mobile permettant aux utilisateurs de découvrir facilement des films.

Les principaux objectifs sont :

- Utiliser une API externe
- Gérer des données locales
- Créer une interface mobile
- Permettre l'interaction utilisateur

3 Environnement développement

3.1 Environnement logiciel

Le développement de l'application Flixty Connect a été réalisé à l'aide de l'environnement de développement Android Studio.

Android Studio permet le développement d'applications Android et propose plusieurs outils facilitant la création de l'interface utilisateur, la gestion du code ainsi que le débogage de l'application.

3.2 Technologies utilisées

Plusieurs technologies ont été utilisées dans le cadre du développement de l'application :

Technologie	Utilisation
Android Studio	Développement de l'application
Java / Kotlin	Développement mobile
SQLite	Base de données locale
GitHub	Gestion de version
API TMDb	Récupération des films

3.3 Gestion de version

Le projet est hébergé sur GitHub afin de faciliter :

- La gestion du code source
- La sauvegarde du projet
- La maintenance de l'application

4 Architecture projet

4.1 Architecture Générale

L'application Flixty Connect repose sur une architecture MVVM (Model - View - ViewModel) combinée avec le pattern Repository.

Cette architecture permet de :

- Séparer les responsabilités
- Faciliter la maintenance
- Améliorer la lisibilité du code
- Faciliter les évolutions futures

L'application est organisée en plusieurs couches principales :

- Interface utilisateur (UI)
- ViewModel
- Repository
- Data (Local + API)
- Network
- Utils

4.2 Structure du Projet

L'application est structurée de la manière suivante :

```
com.example.flixtyConnect
├── api
├── data
│   ├── local
│   │   ├── dao
│   │   ├── database
│   │   └── entity
│   ├── model
│   ├── repository
│   └── store
├── network
├── ui
│   ├── components
│   ├── screens
│   └── theme
├── viewmodel
├── utils
├── worker
└── MainActivity.kt
```

Cette organisation permet de séparer clairement chaque responsabilité de l'application.

4.3 Couche UI

Le dossier ui contient l'ensemble de l'interface utilisateur de l'application.

Il est composé de trois sous-dossiers :

screens

Contient les écrans principaux :

- LoginScreen
- RegisterScreen
- MainScreen
- MovieScreen
- DetailScreen
- FavoritesScreen
- ProfilScreen
- PrincipalScreen

Chaque écran correspond à une fonctionnalité principale de l'application.

components

Contient les composants réutilisables :

- MovieCard
- MovieCarousel
- CommentItem
- AppNavigation
- GenericComponents
- ThemeSwitcher

Ces composants permettent d'éviter la duplication de code.

theme

Contient :

- Colors
- Theme
- Typography

Permet de gérer le thème global de l'application.

4.4 Couche ViewModel

Le dossier viewmodel contient les ViewModels de l'application :

- MovieViewModel
- MovieDetailViewModel
- UserViewModel
- UserFilmViewModel

- CommentaireViewModel
- FriendViewModel
- NotificationViewModel

Le rôle des ViewModels est de :

- Gérer la logique métier
- Communiquer avec les Repository
- Fournir les données à l'interface utilisateur

Cette couche permet de séparer l'interface utilisateur de la logique métier.

4.5 Couche Repository

Le dossier repository contient :

- MovieRepository
- UserRepository
- UserFilmRepository
- CommentaireRepository
- FriendRepository
- NotificationRepository

Les repositories permettent :

- Centraliser l'accès aux données
- Gérer API + Base locale
- Faciliter les tests

4.6 Couche Data

Le dossier data contient la gestion des données.

Local

Contient la base de données locale Room :

entity

- UserEntity
- MovieEntity
- UserFilmEntity
- CommentaireEntity
- FriendEntity

dao

- UserDao
- MovieDao
- UserFilmDao
- CommentaireDao
- FriendDao

database

- AppDatabase

model

Contient les modèles provenant de l'API :

- Movie
- MovieDetails
- Genre
- Video
- ProductionCompany

4.7 store

Contient :

- ThemeDataStore

Utilisé pour stocker les préférences utilisateur.

4.8 Couche Network

Le dossier network contient :

- NetworkModule
- AuthInterceptor

Cette couche gère :

- Gestion API
- Interception requêtes

4.9 Couche Utils

Le dossier utils contient :

- NotificationHelper
- NotificationScheduler
- PermissionUtils
- NotificationPreferences

Permet de centraliser les fonctions utilitaires.

4.10 Worker

Le dossier worker contient :

- ReleaseNotificationWorker

Utilisé pour :

- Notifications planifiées
- Background tasks

4.11 Activité principale

Le fichier MainActivity.kt constitue le point d'entrée de l'application.

Il initialise :

- Navigation
- Thème
- Interface principale

5 Base de données

L'application utilise une base de données locale SQLite afin de stocker les données utilisateurs.

La base de données permet de gérer :

- Utilisateurs
- Favoris
- Notes
- Commentaires

Tables principales

Table User

Cette table permet de stocker les informations relatives aux utilisateurs.

Champs :

- id
- username
- email
- password
- roles
- avatar

Table UserFilm

Cette table permet de gérer les interactions entre utilisateurs et films.

Champs :

- id
- tmdbId
- saved
- rating
- idUser

Table Commentaire

Cette table permet de stocker les commentaires des utilisateurs.

Champs :

- id
- comment
- createdAt
- idUserFilm

5.1 Relations

User -> UserFilm

UserFilm -> Commentaire

6 API utilisées

L'application utilise l'API TMDb (The Movie Database) afin de récupérer les données relatives aux films.

Les informations récupérées sont :

- Liste de films
- Détails films
- Images
- Notes

Exemple endpoints

- /movie/popular
- /movie/upcoming
- /movie/{id}

Les données sont récupérées sous format JSON puis traitées dans l'application.

7 Fonctionnalités techniques

L'application Flixy Connect propose plusieurs fonctionnalités techniques permettant d'assurer le bon fonctionnement de l'application ainsi que l'interaction entre l'utilisateur, l'API externe et la base de données locale.

Ces fonctionnalités reposent sur :

- Architecture MVVM
- Base de données Room
- API TMDb
- WorkManager
- StateFlow
- DataStore

7.1 Authentification Utilisateur

7.1.1 Présentation

L'application FlixyConnect permet aux utilisateurs de créer un compte et de se connecter afin d'accéder aux fonctionnalités personnalisées :

- Profil utilisateur
- Films sauvegardés
- Commentaires
- Statistiques utilisateur

L'authentification repose sur :

- Base de données locale Room
- Hashage sécurisé des mots de passe
- Gestion de session via SharedPreferences
- Architecture MVVM

Technologies utilisées :

- Room
- Bcrypt
- Android SharedPreferences
- MVVM

7.1.2 Structure de la fonctionnalité

Architecture utilisée :

```
UI
├── LoginScreen
├── RegisterScreen

ViewModel
├── UserViewModel

Repository
├── UserRepository

Database
├── AppDatabase
├── UserEntity
├── UserDao
```

7.1.3 Modèle utilisateur (UserEntity)

Le modèle utilisateur est stocké dans la base de données locale Room.

```
@Entity(
    tableName = "users",
    indices = [Index(value = ["email"], unique = true)]
)
data class UserEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val username: String,
    val email: String,
    val password: String,
    val roles: String = "",
    val avatar: String? = null
)
```

Explication

- id : identifiant utilisateur
- username : nom affiché
- email : email unique
- password : mot de passe hashé
- roles : rôle utilisateur
- avatar : image profil

7.1.4 Inscription utilisateur

L'utilisateur peut créer un compte via l'écran RegisterScreen.

Champs requis :

- Email
- Nom utilisateur
- Mot de passe

Validation :

- Vérification email valide
- Vérification champs non vides
- Vérification email unique

3:11

Créer un compte

Remplissez vos informations pour commencer

Adresse email

Entrez votre email

Nom d'utilisateur

Entrez votre nom

Mot de passe

Entrez votre mot de passe

Créer un compte

Déjà un compte ? Connexion

Principal Films Profil

Page d'inscription

Sécurité du mot de passe

Le mot de passe est hashé avec BCrypt avant stockage.

```
val hashedPassword = BCrypt.hashpw(user.password, Bcrypt.gensalt())
```

Puis sauvegarde :

```
val secureUser = user.copy(password = hashedPassword)
repository.saveUser(secureUser)
```

Sauvegarde session utilisateur

Les informations utilisateur sont sauvegardées avec SharedPreferences.

```
val sharedPref = getApplication<Application>()
    .getSharedPreferences("prefs", Context.MODE_PRIVATE)

with(sharedPref.edit()) {
    putString("email", user.email)
    putString("username", user.username)
    putInt("userId", id.toInt())
    apply()
}
```

7.1.5 Connexion utilisateur

L'utilisateur peut se connecter avec :

- Email
- Mot de passe

Processus :

1. Récupération utilisateur
2. Vérification mot de passe
3. Chargement profil
4. Sauvegarde session

3:11

BON RETOUR
Connectez-vous pour continuer

Adresse email

Entrez votre email

Mot de passe

Entrez votre mot de passe

[Mot de passe oublié ?](#)

Se connecter

[Pas encore de compte ?](#) [Créer un compte](#)

Principal Films Profil

Page de connexion

Vérification mot de passe

```
if (user != null && BCrypt.checkpw(password, user.password)) {  
    _currentUser.value = user  
}
```

Navigation après connexion

```
navController.navigate("profile") {  
    popUpTo("login") { inclusive = true }  
}
```

7.1.6 UserViewModel

Le UserViewModel gère la logique métier.

Fonctionnalités :

- register()
- login()
- logout()
- loadUser()
- loadStats()

Code important

Fonction register :

```
fun register(user: UserEntity, onResult: (Boolean) -> Unit)
```

Fonction login :

```
fun login(email: String, password: String, onResult: (Boolean) -> Unit)
```

Fonction logout :

```
fun logout()
```

7.1.7 Gestion session utilisateur

Session stockée dans :

- SharedPreferences
- StateFlow ViewModel

Variables utilisées :

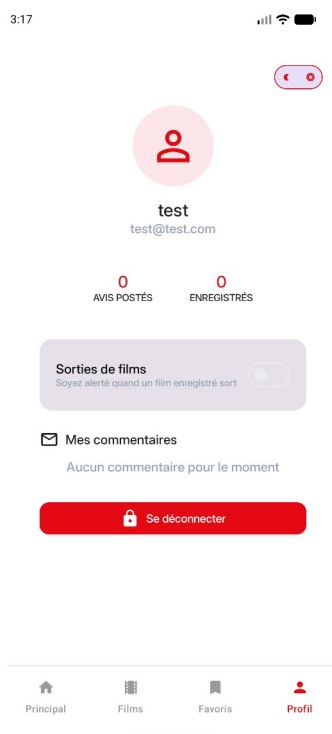
```
private val _currentUser = MutableStateFlow<UserEntity?>(null)  
val currentUser: StateFlow<UserEntity?> = _currentUser
```

7.1.8 Sécurité

Mesures de sécurité mises en place :

- Hash mot de passe Bcrypt
- Email unique
- Validation formulaire
- Session sécurisée

7.2 Profil Utilisateur



Page de profil

7.2.1 Présentation

La page profil permet à l'utilisateur connecté de :

- Voir ses informations personnelles (nom, email, avatar)
- Consulter ses statistiques (avis postés, films enregistrés)
- Gérer ses commentaires
- Activer/désactiver les notifications pour les sorties de films
- Se déconnecter

Technologies et bibliothèques utilisées :

- Jetpack Compose
- Room
- Flow

Architecture : MVVM, avec UserViewModel, MovieViewModel et NotificationViewModel.

7.2.2 Chargement des données

Les données utilisateur sont chargées depuis :

- Room (UserEntity, CommentaireEntity, UserFilmEntity)
- API TMDB (via MovieRepository et GenreRepository pour films et genres)

```
val currentUser by viewModel.currentUser.collectAsState()
val reviewCount by viewModel.reviewCount.collectAsState(initial = 0)
val savedMoviesCount by viewModel.savedMoviesCount.collectAsState(initial = 0)
val comments by viewModel.comments.collectAsState(initial = emptyList())
val userFilms by viewModel.userFilms.collectAsState(initial = emptyList())
```

7.2.3 Interface utilisateur

La page est divisée en plusieurs sections :

1. En-tête : avatar, nom, email
2. Statistiques : avis postés, films enregistrés
3. Notifications : toggle pour les sorties de films
4. Commentaires : liste des commentaires avec possibilité de suppression
5. Déconnexion : bouton pour se déconnecter

```
Card(
    shape = RoundedCornerShape(16.dp),
    colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
    modifier = Modifier.fillMaxWidth()
) {
    Row(
        modifier = Modifier.padding(20.dp).fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceEvenly
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Text(reviewCount.toString(), color =
MaterialTheme.colorScheme.primary, fontSize = 20.sp)
            Text("AVIS POSTÉS", fontSize = 12.sp)
        }
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Text(savedMoviesCount.toString(), color =
MaterialTheme.colorScheme.primary, fontSize = 20.sp)
            Text("ENREGISTRÉS", fontSize = 12.sp)
        }
    }
}
```

7.2.4 Gestion des commentaires

Chaque commentaire est lié à un film enregistré. L'utilisateur peut :

- Voir le titre du film
- Lire son commentaire
- Supprimer un commentaire

```
comments.forEach { comment ->
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 6.dp)
            .clickable { /* navigate to movie detail */ },
        shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
    ) {
        Row(
            modifier = Modifier.padding(16.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(modifier = Modifier.weight(1f)) {
                Text(text = movieTitleMap[comment.userFilmId] ?: "Film
inconnu")
                Spacer(modifier = Modifier.height(4.dp))
                Text(text = comment.comment, maxLines = 2, overflow =
TextOverflow.Ellipsis)
            }
            IconButton(onClick = { viewModel.deleteComment(comment,
currentUser?.id ?: return@IconButton)
        }) {
                Icon(Icons.Default.Delete, contentDescription = "Supprimer",
tint = MaterialTheme.colorScheme.error)
            }
        }
    }
}
```

7.2.5 Notifications

L'utilisateur peut activer ou désactiver les notifications pour :

- Sorties de films enregistrés

La permission `POST_NOTIFICATIONS` est demandée si nécessaire.

```
Switch(  
    checked = releaseEnabled,  
    onCheckedChange = { enabled →  
        if (enabled) {  
            if (hasNotificationPermission(context)) {  
                notifViewModel.toggleRelease(true)  
            } else {  
                permissionLauncher.launch(android.Manifest.permission.POST_NOTIFICATIONS)  
            }  
        } else {  
            notifViewModel.toggleRelease(false)  
        }  
    }  
)
```

7.2.6 Déconnexion

Le bouton Se déconnecter réinitialise la session et navigue vers la page login.

```
Button(onClick = {  
    logout(sharedPreferences, viewModel)  
    navController.navigate("login") { popUpTo("profile") { inclusive = true } }  
}) {  
    Icon(Icons.Default.Lock, contentDescription = null)  
    Spacer(modifier = Modifier.width(8.dp))  
    Text("Se déconnecter")  
}
```

7.2.7 Observations techniques

- Les données sont réactives via `StateFlow`
- `MVVM` permet de séparer UI et logique métier
- L'intégration `MovieViewModel` et `NotificationViewModel` montre la communication `inter-ViewModel`
- Les cartes et toggles utilisent `Jetpack Compose Material3`

7.3 Gestion des films et API TMDb

7.3.1 Présentation

L'application récupère les films depuis l'API TMDb afin de proposer à l'utilisateur :

- Les films actuellement à l'affiche (now_playing)
- Les films populaires
- Les films top-rated
- Les films futurs (upcoming)

La récupération des films repose sur :

- Retrofit
- Kotlinx Serialization
- Intercepteur AuthInterceptor pour l'authentification API

Technologies et librairies utilisées :

- Retrofit
- OkHttp
- Kotlinx Serialization
- MVVM

7.3.2 Architecture de la fonctionnalité

```
UI
├── MovieListScreen
├── MovieDetailScreen
ViewModel
├── MovieViewModel
Repository
├── MovieRepository
Network
├── NetworkModule
├── MovieApi
├── AuthInterceptor
Database
├── MovieEntity
├── MovieDao
```

7.3.3 Modèle Film (MovieEntity)

Le modèle film est stocké en base locale Room pour permettre l'affichage offline et les interactions utilisateurs :

```
@Entity(tableName = "movies")
data class MovieEntity(
    @PrimaryKey val id: Int,
    val title: String,
    val overview: String,
    val posterPath: String?,
    val backdropPath: String?,
    val releaseDate: String,
    val voteAverage: Double
)
```

Explications :

- id : identifiant unique
- title : titre du film
- overview : synopsis
- posterPath / backdropPath : images
- releaseDate : date de sortie
- voteAverage : note moyenne TMDb

7.3.4 Récupération des films via l'API

- L'API TMDb est appelée via Retrofit et NetworkModule.
- Exemple de route now_playing :

```
@GET("3/movie/now_playing")
suspend fun getNowPlaying(@Query("page") page: Int = 1): MovieResponse
```

- AuthInterceptor ajoute automatiquement le token Bearer et le header accept: application/json.
- Les données JSON sont converties en MovieEntity via des extensions ou mappers.

Exemple de transformation JSON → Entity :

```
fun MovieResponse.MovieResult.toEntity() = MovieEntity(
    id = id,
    title = title,
    overview = overview,
    posterPath = poster_path,
    backdropPath = backdrop_path,
    releaseDate = release_date,
    voteAverage = vote_average
)
```

7.3.5 Affichage des films

- Films affichés dans MovieListScreen via LazyColumn.
- Chaque carte film montre :
 - Image
 - Titre
 - Date de sortie
 - Note
- Détail film (MovieDetailScreen) permet de :
 - Lire le synopsis
 - Ajouter aux films sauvegardés
 - Accéder aux commentaires

7.4 Notifications planifiées (WorkManager)

7.4.1 Présentation

L'application notifie l'utilisateur à la sortie d'un film enregistré via WorkManager.

Fonctionnalités :

- Notifications pour les sorties de films
- Activation/désactivation via profil utilisateur
- Gestion des permissions Android 13+

Technologies utilisées :

- WorkManager
- NotificationCompat
- DataStore / SharedPreferences
- MVVM

7.4.2 Architecture

```
UI
├── ProfileScreen (toggle notifications)
ViewModel
├── NotificationViewModel
Worker
├── ReleaseNotificationWorker
Scheduler
├── NotificationScheduler
Utils
├── NotificationHelper
├── NotificationPreferences
├── NotificationPermissionChecker
```

7.4.3 Worker et Notification

ReleaseNotificationWorker : envoie la notification pour un film :

```
val notification = NotificationCompat.Builder(applicationContext,
"releases_channel")
    .setContentTitle("Sortie du film 🎬 ")
    .setContentText("$title est disponible !")
    .setSmallIcon(android.R.drawable.ic_dialog_info)
    .build()
NotificationManagerCompat.from(applicationContext)
    .notify(System.currentTimeMillis().toInt(), notification)
```

7.4.4 Scheduler (NotificationScheduler)

- Planifie le Worker selon releaseDate :

```
val delay = date.time - System.currentTimeMillis()
val request = OneTimeWorkRequestBuilder<ReleaseNotificationWorker>()
    .setInitialDelay(delay, TimeUnit.MILLISECONDS)
    .setInputData(workDataOf("title" to title))
    .addTag("movie_$movieId")
    .build()
WorkManager.getInstance(context).enqueue(request)
```

- Permet aussi d'annuler un Worker si l'utilisateur désactive la notification :

```
WorkManager.getInstance(context).cancelAllWorkByTag("movie_$movieId")
```

7.4.5 Gestion des préférences

- Les préférences sont stockées dans NotificationPreferences (DataStore ou SharedPreferences).
- L'utilisateur peut activer/désactiver les notifications :

```
Switch(
    checked = releaseEnabled,
    onCheckedChangeListener = { enabled →
        if (enabled) {
            if (hasNotificationPermission(context)) {
                notifViewModel.toggleRelease(true)
            } else {
                permissionLauncher.launch(android.Manifest.permission.POST_NOTIFICATIONS)
            }
        } else {
            notifViewModel.toggleRelease(false)
        }
    }
)
```

7.4.6 Ajout d'un nouveau type de notification

Pour créer de nouvelles notifications (ex : promos, événements) :

1. Ajouter une nouvelle clé dans NotificationPreferences.
2. Créer un Worker spécifique si nécessaire.
3. Ajouter un Scheduler dans NotificationScheduler.
4. Ajouter le toggle dans le profil utilisateur.

8 Schémas techniques

Cette partie regroupe les différents schémas techniques du projet Flixy Connect, permettant de visualiser l'organisation et le fonctionnement de l'application. Ces schémas aident à comprendre les relations entre les composants, la structure des données et l'architecture globale.

8.1 Diagramme UML

Afin de représenter l'architecture de l'application Flixy Connect, un diagramme UML a été réalisé à l'aide de l'outil PlantUML. Ce diagramme permet de visualiser les différentes classes de l'application ainsi que les relations entre celles-ci.

Le diagramme UML met en évidence les principales entités du projet :

- Utilisateur
- Films
- Favoris
- Commentaires

Certaines parties du diagramme ont été entourées en rouge afin d'identifier les éléments prévus pour une future évolution de l'application. Ces éléments ne sont pas implémentés dans la version actuelle mais permettent d'anticiper l'évolution de l'architecture.

Le diagramme UML complet est présenté ci-dessous.

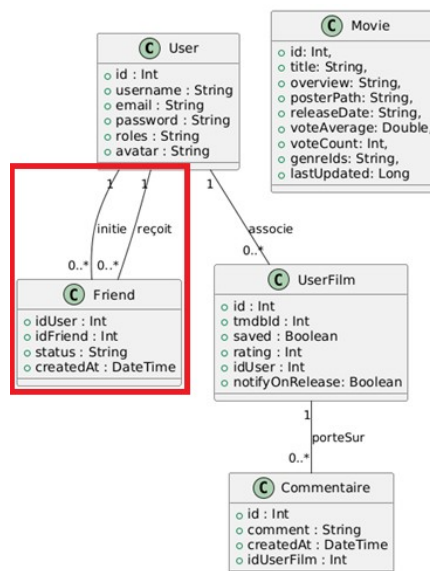


Diagramme UML Flixy Connect

Code PlantUML

Le diagramme UML a été généré à partir du code suivant :

```
@startuml
class User {
    +id : Int
    +username : String
    +email : String
    +password : String
    +roles : String
    +avatar : String
}

class Friend {
    +idUser : Int
    +idFriend : Int
    +status : String
    +createdAt : DateTime
}

class UserFilm {
    +id : Int
    +tmdbId : Int
    +saved : Boolean
    +rating : Int
    +idUser : Int
    +notifyOnRelease: Boolean
}

class Commentaire {
    +id : Int
    +comment : String
    +createdAt : DateTime
    +idUserFilm : Int
}

class Movie {
    +id: Int,
    +title: String,
    +overview: String,
    +posterPath: String,
    +releaseDate: String,
    +voteAverage: Double,
    +voteCount: Int,
    +genreIds: String,
    +lastUpdated: Long
}

' Associations
User "1" -- "0..*" Friend : initie
User "1" -- "0..*" Friend : reçoit
User "1" -- "0..*" UserFilm : associe
UserFilm "1" -- "0..*" Commentaire : porteSur
@enduml
```

8.2 Diagramme base de données

Le diagramme de la base de données montre la structure des tables locales sur le téléphone :

- User : informations sur l'utilisateur.
- UserFilm : association entre un utilisateur et ses films favoris ou notés.
- Commentaire : commentaires laissés par l'utilisateur sur ses films.

Ces schémas facilitent la compréhension de la conception et permettent de mieux appréhender le fonctionnement interne de l'application.

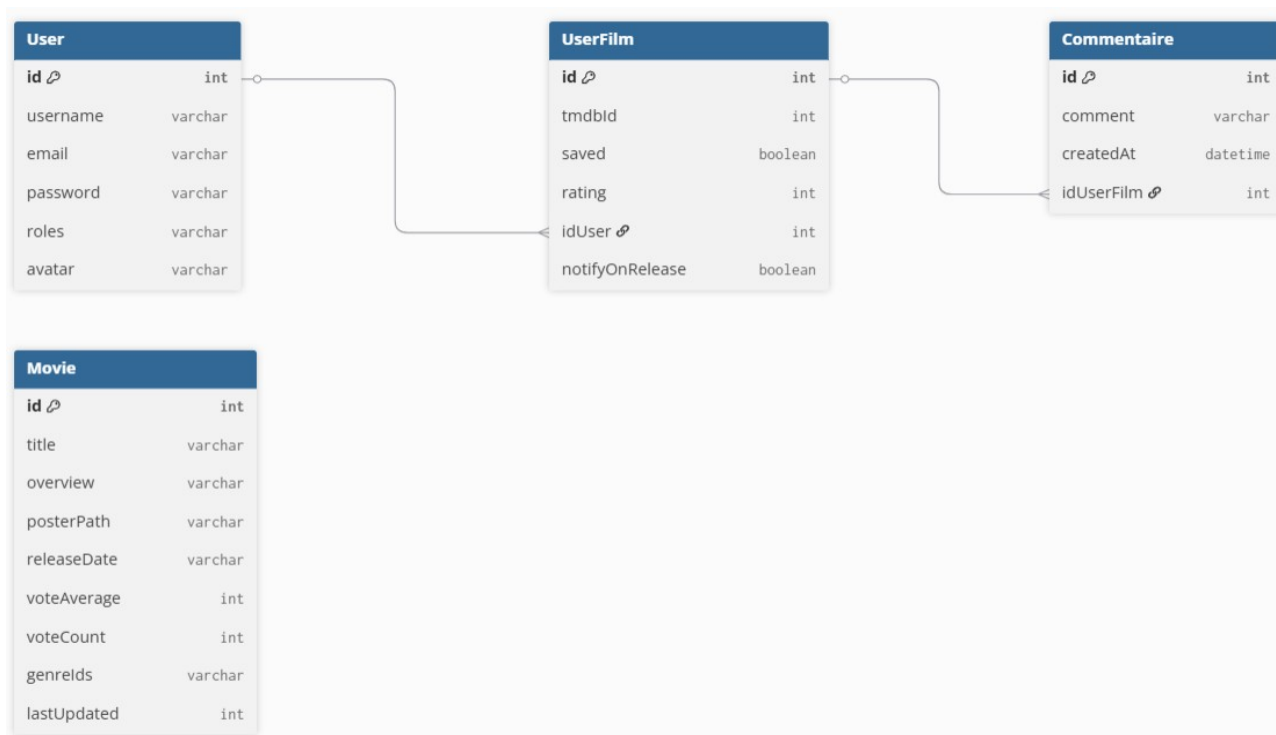


Diagramme de la base de données

Code dbdiagram.io

```
Table User {
  id int [pk, increment]
  username varchar
  email varchar
  password varchar
  roles varchar
  avatar varchar
}

Table UserFilm {
  id int [pk, increment]
  tmdbId int
  saved boolean
  rating int
  idUser int
  notifyOnRelease boolean
}

Table Commentaire {
  id int [pk, increment]
  comment varchar
  createdAt datetime
  idUserFilm int
}

Table Movie {
  id int [pk, increment]
  title varchar
  overview varchar
  posterPath varchar
  releaseDate varchar
  voteAverage int
  voteCount int
  genreIds varchar
  lastUpdated int
}

Ref: User.id < UserFilm.idUser
Ref: UserFilm.id < Commentaire.idUserFilm
```

9 Gestion erreurs

Afin d'assurer la stabilité de l'application Flixy Connect, plusieurs mécanismes de gestion des erreurs ont été mis en place.

9.1 Gestion des erreurs API

L'application utilise une classe `ApiResponse` permettant de gérer les réponses de l'API :

- Success : données récupérées
- Error : erreur lors de la requête

Exemple dans le `MovieViewModel` :

```
when (moviesResult) {  
    is ApiResponse.Success -> _state.update { it.copy(movies = moviesResult.data,  
loading = false) }  
    is ApiResponse.Error -> _state.update { it.copy(error = moviesResult.message,  
loading = false) }  
}
```

Cela permet :

- D'éviter les crashes
- D'afficher un message d'erreur
- D'améliorer l'expérience utilisateur

9.2 Gestion des exceptions

Les appels API sont également protégés avec un `try / catch` afin d'intercepter les erreurs réseau ou système :

```
try {  
    val moviesResult = repo.fetchMovies()  
} catch (e: Exception) {  
    _state.update { it.copy(error = e.message, loading = false) }  
}
```

Cela permet :

- Gérer les erreurs réseau
- Éviter les crashes
- Maintenir l'application stable

9.3 Gestion des états de chargement

L'application gère également les états :

- Loading
- Success
- Error

Grâce à **MovieUIState** :

```
val loading: Boolean = false
val error: String? = null
```

Cela permet :

- Afficher un loader
- Afficher message erreur
- Améliorer UX

9.4 Gestion des erreurs d'authentification

Dans le `UserViewModel`, les erreurs de connexion sont gérées :

- Mauvais mot de passe
- Utilisateur inexistant
- Email déjà utilisé

Exemple :

```
if (user != null && BCrypt.checkpw(password, user.password)) {
```

Sinon :

```
onResult(false)
```

9.5 Gestion globale des erreurs

Les mécanismes suivants sont utilisés :

- `ApiResponse`
- Try / Catch
- `StateFlow`
- Vérification utilisateur
- Gestion loading / error

10 Sécurité

10.1 Sécurisation des données utilisateur

Dans l'application Flixty Connect, plusieurs mécanismes ont été mis en place afin d'assurer la sécurité des données utilisateur, notamment lors de l'inscription et de la connexion.

L'un des principaux mécanismes de sécurité mis en place est le hachage des mots de passe. Lors de l'inscription, le mot de passe de l'utilisateur n'est jamais stocké en clair dans la base de données. Il est d'abord chiffré à l'aide de la bibliothèque BCrypt, puis enregistré sous forme sécurisée.

Exemple utilisé dans **UserViewModel** :

```
val hashedPassword = BCrypt.hashpw(user.password, Bcrypt.gensalt())  
val secureUser = user.copy(password = hashedPassword)
```

Grâce à ce mécanisme :

- Le mot de passe n'est jamais stocké en clair
- Même en cas d'accès à la base de données, les mots de passe restent protégés
- Chaque mot de passe possède un salt unique

10.2 Vérification sécurisée lors de la connexion

Lors de la connexion, le mot de passe saisi par l'utilisateur est comparé avec le mot de passe haché stocké dans la base de données à l'aide de **BCrypt.checkpw()** :

```
if (user != null && BCrypt.checkpw(password, user.password)) {
```

Cette méthode permet :

- Une comparaison sécurisée
- Une protection contre les attaques par brute force
- Une authentification fiable

10.3 Stockage local sécurisé

Une fois connecté, les informations utilisateur sont stockées localement dans les SharedPreferences :

```
val sharedPref = getApplication<Application>()
    .getSharedPreferences("prefs", Context.MODE_PRIVATE)
```

Les données stockées sont :

- Email utilisateur
- Username
- ID utilisateur

Le mode MODE_PRIVATE garantit que seules l'application peut accéder à ces données.

10.4 Architecture MVVM et Sécurité

L'application utilise une architecture MVVM (Model View ViewModel) qui améliore également la sécurité du projet.

Cette architecture permet :

- Une séparation claire des responsabilités
- Une meilleure gestion des données
- Une limitation de l'accès direct à la base de données

Dans cette architecture :

- La View ne communique jamais directement avec la base de données
- Le ViewModel agit comme intermédiaire sécurisé
- Les Repositories contrôlent l'accès aux données

Architecture sécurisée :

UI -> ViewModel -> Repository -> Database

Cette séparation permet :

- D'éviter les accès non contrôlés
- De centraliser la logique de sécurité
- De faciliter la maintenance

10.5 Gestion de session utilisateur

L'application gère également la session utilisateur via :

- currentUser (StateFlow)
- SharedPreferences

Exemple :

```
private val _currentUser = MutableStateFlow<UserEntity?>(null)
```

Cela permet :

- Gestion sécurisée de l'utilisateur connecté
- Mise à jour automatique de l'interface
- Protection contre accès non autorisés

10.6 Sécurité Base de Données Locale

L'application utilise Room Database, ce qui apporte :

Requêtes typées

Protection contre injection SQL

Gestion sécurisée des données locales

Les accès aux données sont réalisés via :

- DAO
- Repository
- ViewModel

Cela renforce la sécurité globale de l'application.

10.7 Sécurité globale mise en place

Plusieurs niveaux de sécurité ont été mis en place :

- Hachage des mots de passe avec Bcrypt
- Architecture MVVM
- Accès sécurisé à la base de données
- Gestion de session utilisateur
- SharedPreferences sécurisées
- Repository pattern

11 Limites

Malgré les fonctionnalités développées, certaines limites techniques sont présentes dans la version actuelle de Flixty Connect :

- Données stockées localement : les films favoris, les notes et commentaires sont conservés uniquement sur le téléphone de l'utilisateur.
- Pas de synchronisation serveur : la base locale n'est pas synchronisée avec un serveur central, ce qui limite l'accès aux données depuis plusieurs appareils.

Ces limites sont liées à la version initiale de l'application, mais elles pourront être corrigées dans des évolutions futures.

12 Évolutions

Dans le cadre des futures versions de l'application Flixy Connect, plusieurs améliorations techniques et fonctionnelles pourront être envisagées afin d'enrichir l'expérience utilisateur et d'améliorer les performances globales de l'application.

12.1 Système d'amitiés entre utilisateurs

Une première évolution consisterait à intégrer un système d'amitiés entre utilisateurs. Cette fonctionnalité permettrait aux utilisateurs d'ajouter d'autres membres comme amis afin de créer une dimension plus sociale au sein de l'application.

Cette fonctionnalité pourrait inclure :

- Recherche d'utilisateurs
- Ajout d'amis
- Suppression d'amis
- Liste d'amis
- Consultation du profil d'un ami

La mise en place de ce système nécessiterait l'ajout :

- D'une nouvelle table "Friend" ou "UserFriend"
- De nouvelles interfaces utilisateur
- De nouvelles requêtes dans la base de données

Cette évolution permettrait d'améliorer l'interaction entre utilisateurs et d'enrichir l'expérience globale.

12.2 Partage de contenus entre amis

Dans la continuité du système d'amitiés, une fonctionnalité de partage de films entre amis pourrait être ajoutée.

Les utilisateurs pourraient :

- Partager un film avec leurs amis
- Voir les films recommandés par leurs amis
- Consulter les films favoris de leurs amis

Cette évolution permettrait de rendre l'application plus interactive et dynamique.

12.3 Notifications

Une autre amélioration consisterait à ajouter un système de notifications afin d'informer l'utilisateur de certaines actions.

Exemples :

- Nouveau film disponible
- Nouveau commentaire
- Demande d'ami
- Film recommandé
- Synchronisation des données

12.4 Synchronisation des données

Actuellement, les données sont stockées localement sur le terminal. Une évolution possible serait la mise en place d'une synchronisation avec un serveur distant.

Cette évolution permettrait :

- Sauvegarde des données
- Synchronisation multi-appareils
- Récupération du compte sur un autre appareil

13 Conclusion

Le développement de l'application Flixy Connect a permis de proposer une solution mobile moderne et fonctionnelle, répondant aux besoins initiaux de Flixy Studios en matière de découverte et de gestion de contenus cinématographiques.

Grâce à cette application, les utilisateurs peuvent facilement consulter un catalogue de films, accéder à des informations détaillées, sauvegarder leurs films favoris et interagir avec le contenu via des commentaires et des évaluations. L'interface a été conçue pour être intuitive et accessible, garantissant une expérience fluide même pour des utilisateurs novices.

Cette première version constitue une base solide, offrant un ensemble de fonctionnalités essentielles centrées sur la découverte et la gestion des films. Les évolutions futures, telles que l'intégration d'un système social basé sur les amitiés et le partage de contenus, permettront d'enrichir davantage l'expérience utilisateur et de renforcer l'engagement au sein de l'application.

En résumé, Flixy Connect pose les fondations d'une solution évolutive, capable de s'adapter aux besoins des utilisateurs et de s'améliorer progressivement tout en offrant une expérience simple, efficace et agréable.